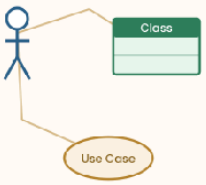
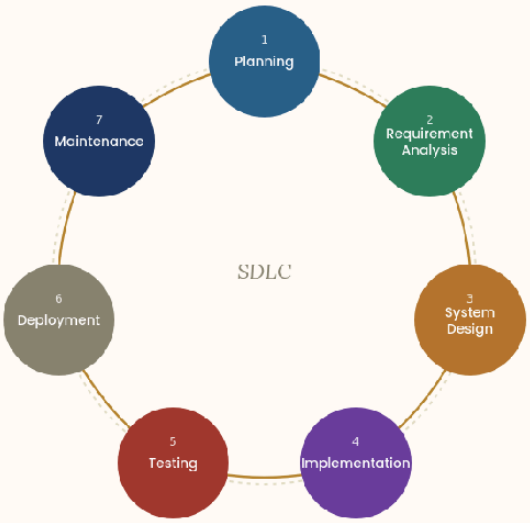


NVQ LEVEL 5

System Analysis & Design

SDLC, Use Case, ER, Class & Data Flow Diagrams

PROJECT **Library Automation System**



Farook Musarraf

1. Introduction

This report presents the System Analysis and Design (SAD) evidence for the Library Automation System project. It covers the Software Development Life Cycle (SDLC) methodologies available, the methodology chosen for this project and why, all phases of the SDLC as applied to this specific system, and the supporting design diagrams: a Use Case diagram, an Entity-Relationship diagram, a Class diagram, and Data Flow Diagrams (context level and level 1).

2. SDLC — Development Methodologies

The Software Development Life Cycle (SDLC) is the structured process a team follows to plan, build, test and maintain software. Several established models exist, each suited to different project shapes:

2.1 Waterfall Model

A strictly sequential model: Requirements → Design → Implementation → Testing → Deployment → Maintenance, with each phase completed before the next begins. Simple to manage and document, but inflexible — a mistake or missed requirement discovered late is expensive to fix, since there is no return to an earlier phase.

2.2 Incremental Model

The system is broken into functional increments (e.g. "the database", then "the core business logic", then "the API", then "the UI"), each one analyzed, designed, built and tested as a small working unit before the next increment starts. Requirements can be refined between increments based on what was learned building the previous one.

2.3 Iterative / Agile (Scrum-style)

Work is planned and delivered in short, fixed cycles (sprints), with working software reviewed at the end of each cycle and the plan adjusted based on feedback. Well suited to projects where requirements may evolve, at the cost of needing more active, ongoing planning than Waterfall.

2.4 Spiral Model

Combines iterative development with explicit risk analysis at the start of every cycle (identify risks → evaluate alternatives → build/test → plan next cycle). Favoured for large, high-risk projects where the cost of an undetected risk is severe; the overhead of formal risk analysis is usually not justified for a small project.

2.5 V-Model

An extension of Waterfall that pairs each development phase with a corresponding testing phase planned at the same time (e.g. requirements are paired with acceptance testing, design with system testing). Improves test coverage and traceability, but inherits Waterfall's inflexibility to changing requirements.

2.6 Rapid Application Development (RAD)

Prioritizes fast, prototype-driven development with heavy user/client involvement over extensive up-front planning and documentation — well suited to smaller systems with clear scope and a single stakeholder in the loop, less suited to large teams or systems with strict regulatory documentation needs.

3. Methodology Selected for This Project

The Incremental Model was selected. This project was, in practice, built exactly this way: the relational database schema (Database Systems unit) was completed and validated first as a working increment on its own; the Java business logic and REST API (Software Programming unit) formed the second increment, built directly on the first and validated with its own automated test suite (Software Testing unit) before moving on; the web front-end (Web Programming unit) was

the third increment, consuming the already-working API. Each increment was independently runnable and testable, which is the defining characteristic of the Incremental Model and is what lets defects (such as a loan-ID bug found while testing the second increment) be caught and fixed without disturbing the increments already completed.

This was preferred over:

- Waterfall — a single "big bang" integration at the end would have made it far harder to isolate the loan-ID bug that automated testing actually caught mid-project.
- Spiral — the project's risk profile (a course project with a known, bounded scope) does not justify the overhead of formal risk-analysis cycles.
- RAD — appropriate for very small, prototype-only systems; this project needed proper layered design and a persistent data model, which RAD's rapid-prototyping emphasis is not built around.

4. SDLC Steps Applied to This Project

4.1 Planning / Feasibility Study

Problem identified: manual, paper-based tracking of book lending at a library branch is slow and error-prone — staff cannot easily tell which copies are available, which loans are overdue, or how much a member owes in fines.

- Technical feasibility: achievable entirely with freely available, already-installed tools — Java (JDK), a relational database, and a browser — no paid licenses or specialist hardware required.
- Economic feasibility: zero additional cost; the JDK's own HTTP server removed the need for a paid or complex application server.
- Schedule feasibility: scoped to fit the unit-by-unit structure of the qualification, one increment per unit.

4.2 Requirement Analysis

Functional requirements (see also the Use Case diagram, Section 5.1):

- Register, search, update and delete member records.
- Add, search, update and delete book records, tracking multiple physical copies per title.
- Issue a loan against a member and a specific book, enforcing eligibility rules.
- Return a loan and automatically calculate any overdue fine.
- List currently overdue loans and send automated email notifications.

Non-functional requirements:

- The system must reject invalid operations (e.g. borrowing with no copies left) with a clear error rather than corrupting data.
- Business rules must be independently testable without a browser or live database (met by the layered architecture and the 19-case automated test suite).
- The backend must run with only the JDK installed — no external application server or paid dependency.

4.3 System Design

Covered in full in Section 5: the Use Case diagram models who does what; the ER diagram models how data is structured relationally; the Class diagram models the object-oriented domain model; the Data Flow Diagrams model how information moves through the system's processes.

4.4 Implementation

The design was implemented in Java as a layered application (model / repository / policy / service / API, following SOLID principles — see the Software Programming report) exposing a REST API, with a database schema implemented in MySQL and a browser front-end consuming the API.

4.5 Testing

Verified with a 19-case automated unit test suite covering member/book CRUD, loan issuing rules, and fine calculation (including boundary cases), plus manual end-to-end testing of every REST endpoint with curl. A deliberately injected fault (wrong fine rate) was used to confirm the test suite actually detects regressions — see the Software Testing report for full detail.

4.6 Deployment

Deployed by compiling with `javac` and running the resulting API server locally (`java -cp bin com.library.api.ApiServer`), which also serves the front-end's static files from the same origin — no separate web server or deployment pipeline needed for this scale of system.

4.7 Maintenance

The layered, SOLID-based design directly supports maintenance: a new fine policy, a new loan-length rule, or a persistent JDBC-backed repository (replacing the in-memory one) can each be added as a new class without modifying existing, already-tested code — see the Open/Closed and Dependency Inversion discussion in the Software Programming report.

5. System Design

5.1 Use Case Diagram

Two actors interact with the system: the Librarian (primary actor, performing every day-to-day operation) and an automated Notification Service (secondary actor, representing the email-sending mechanism). Issue Loan includes Check Borrowing Eligibility; Return Loan includes Calculate Overdue Fine; both Issue Loan and Return Loan include Send Email Notification, since each triggers an automated message to the member.

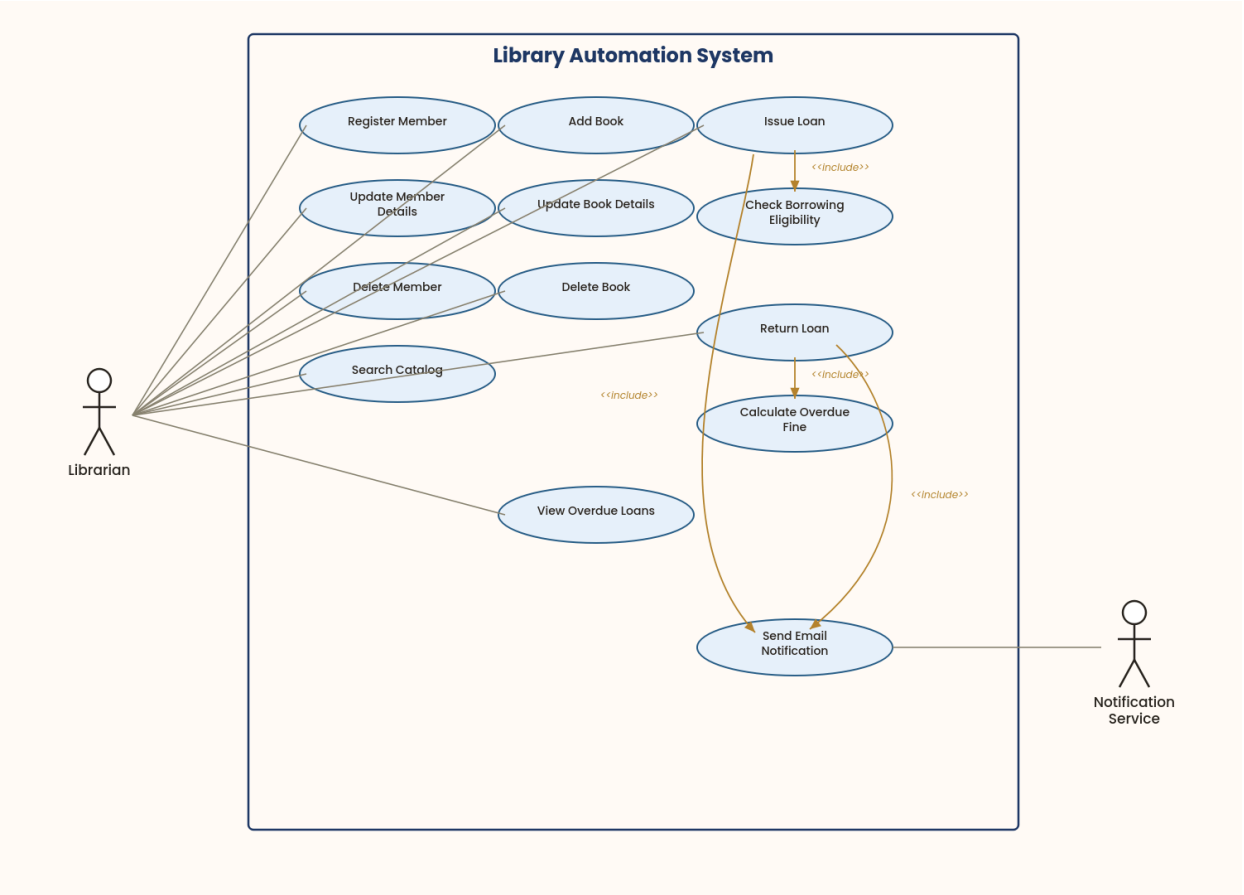


Figure 1: Use Case diagram for the Library Automation System.

5.2 Entity-Relationship (ER) Diagram

The full relational schema (10 tables, normalized to 3NF) is developed in detail in the Database Systems report; it is reproduced here as it is a core System Design artifact. Crow's-foot notation is used: an arrow with a crow's foot marks the 'many' side of a relationship.

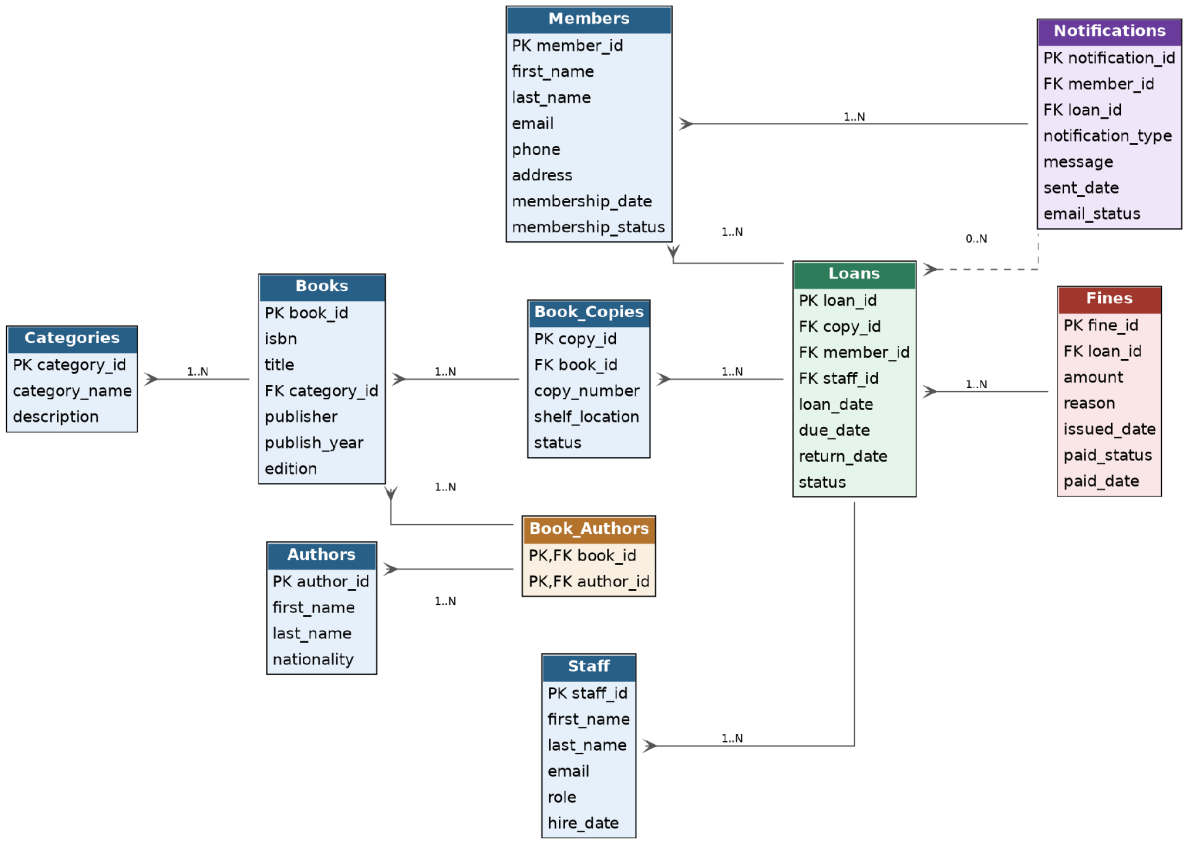


Figure 2: Entity-Relationship diagram for the Library Automation System database.

5.3 Class Diagram

This is a domain-level class diagram: it models the real-world entities the system manages (Member, Staff, Book, Author, Category, Loan, Fine, Notification), their attributes, key behaviours, and the multiplicity of relationships between them. It is intentionally a design-level view of the problem domain rather than the implementation's internal software architecture (interfaces, repositories, dependency injection), which is covered separately in the Software Programming report's SOLID-principles class diagram.

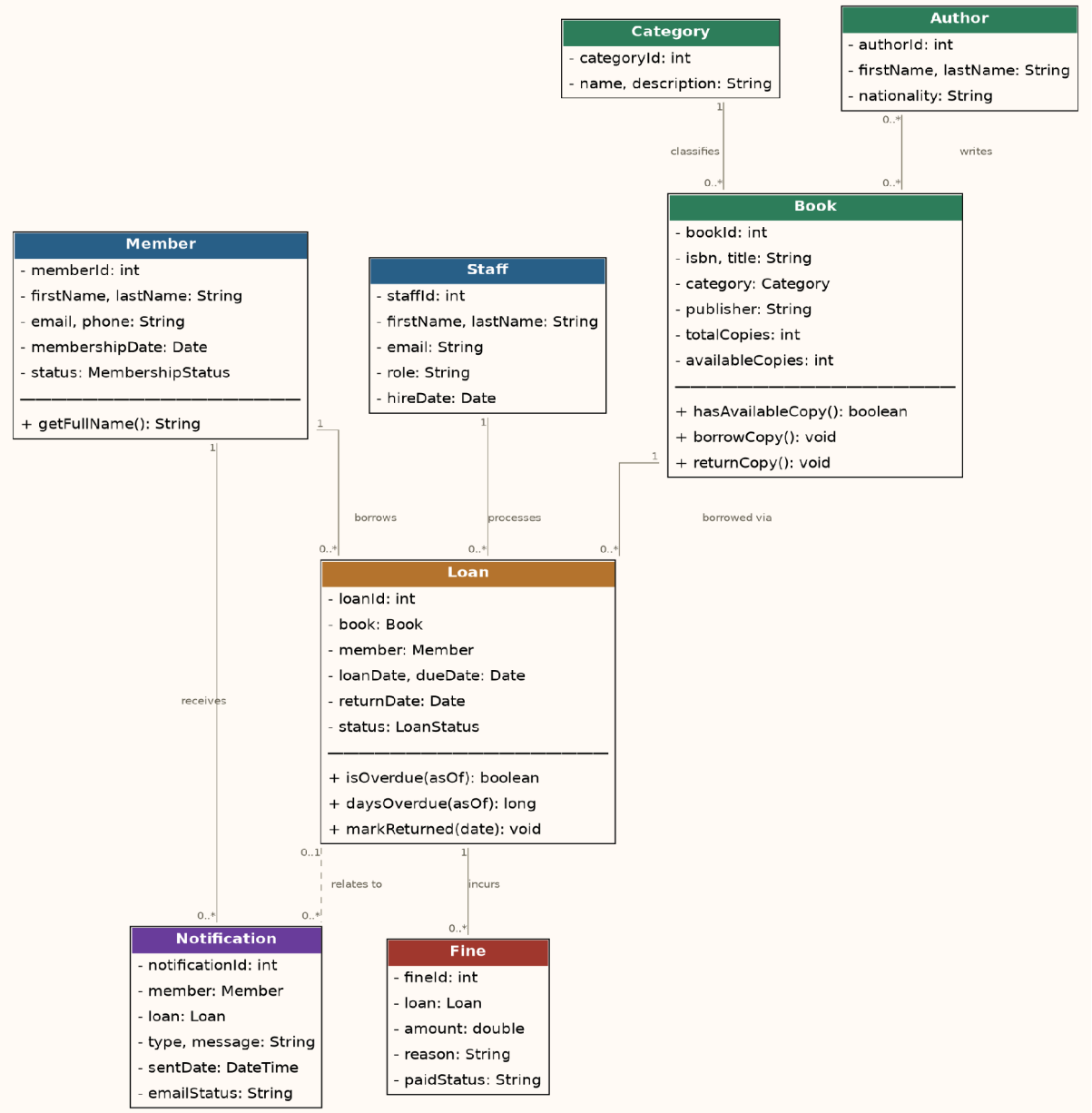


Figure 3: Domain class diagram for the Library Automation System.

5.4 Data Flow Diagrams (DFD)

5.4.1 Context Diagram (Level 0)

The whole system is treated as a single process, bounded by its two external entities: the Librarian, who supplies member/book details and loan/return requests and receives confirmations and reports back; and the Member, who receives automated email notifications generated by the system.

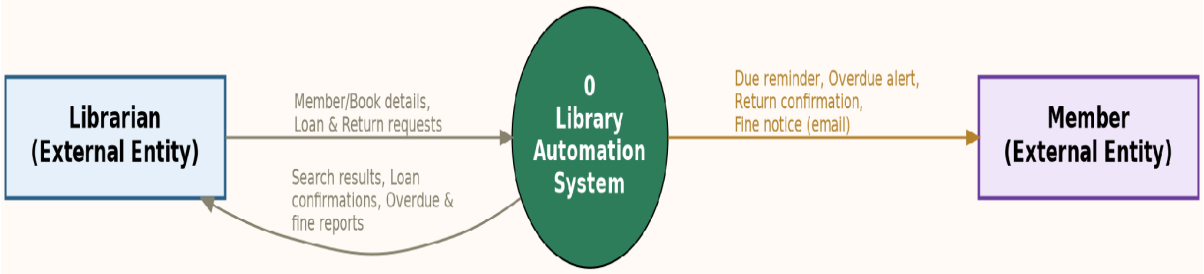


Figure 4: Context-level (Level 0) Data Flow Diagram.

5.4.2 Level 1 DFD

The single process is decomposed into four sub-processes, each reading from and writing to its own data store (matching the Database Systems tables): 1.0 Manage Members, 2.0 Manage Books, 3.0 Manage Loans, and 4.0 Generate Notifications. Manage Loans reads from the Members and Books stores to check eligibility before writing a new Loans record, and raises an event that 4.0 turns into an outbound email.

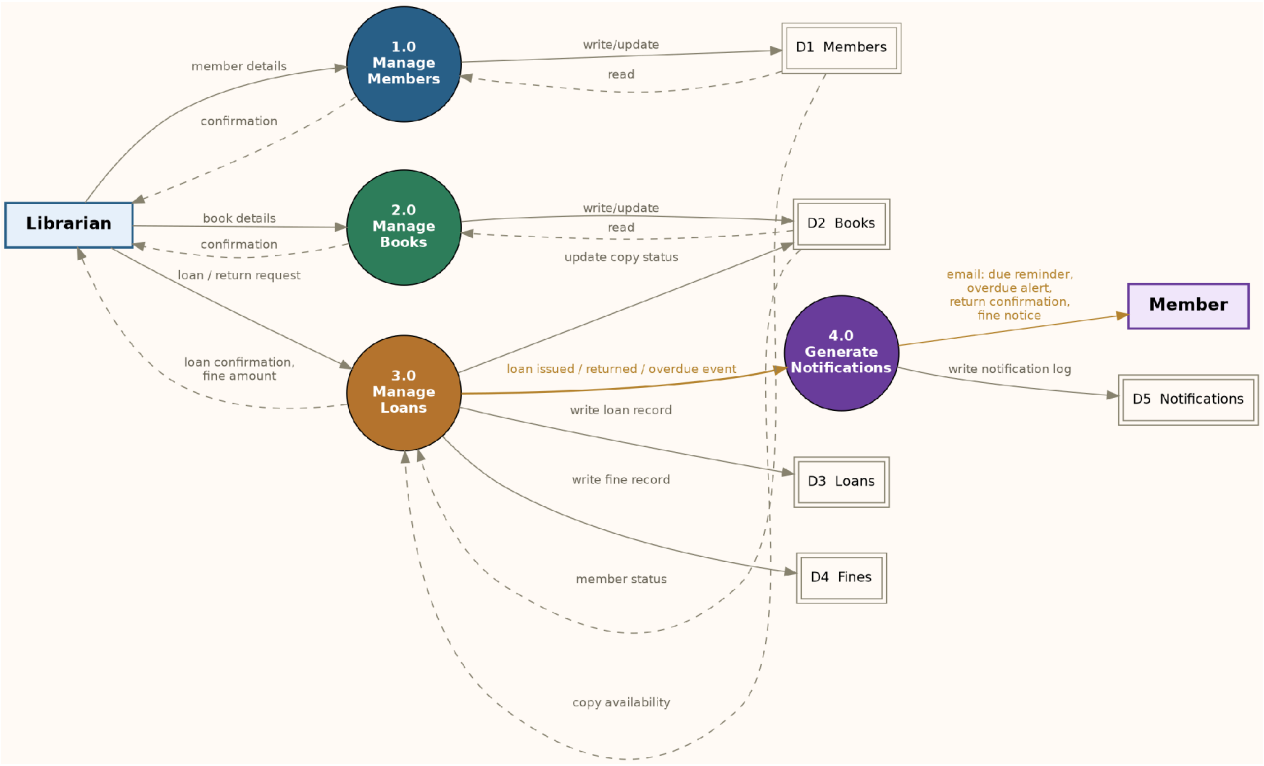


Figure 5: Level 1 Data Flow Diagram, decomposing the system into its four core processes.

6. Conclusion

This report has covered the SDLC methodologies available, justified the Incremental Model as the methodology actually used to build the Library Automation System, walked through all seven SDLC phases as applied to this project, and presented the four core System Design artifacts — Use Case, ER, Class and Data Flow Diagrams — that together specify what the system does, how its data is structured, how its domain objects relate, and how information flows between its processes. Together with the Database Systems, Software Programming and Software Testing reports, this gives a complete analysis-through-implementation picture of the project.